

Chapter 3.2

Platforms, Themes, and Design for Everyone

Contents

Colors

Material Design and Themes

Design for Everyone

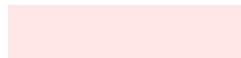
Colors

Colors

- Color and ColorSwatch constants which represent Material design color palette.
- Most swatches have colors from 100 to 900 in increments of one hundred, plus the color 50.
- The smaller the number, the more pale the color. The greater the number, the darker the color.
- The accent swatches (e.g. redAccent) only have the values 100, 200, 400, and 700.

Color Class

```
Color c1 = const Color(0xFFCE4EC);
```



```
Color c2 = const Color.fromARGB(0xFF, 0x42, 0xA5, 0xF5);
```

```
Color c3 = const Color.fromARGB(255, 66, 165, 245);
```

```
Color c4 = const Color.fromRGBO(66, 165, 245, 1.0);
```

Colors

Select a specific color from one of the swatches:

```
// Selects a mid-range green.  
Color selection = Colors.green[400]!;
```

Each ColorSwatch constant is a color and can used directly:

```
Container(  
    // same as Colors.blue[500] or Colors.blue.shade500  
    color: Colors.blue,  
)
```

ColorSwatch

ColorSwatch is a class that represents a collection of colors related to a single color theme.

It's useful for creating color palettes that vary in shades, tints, and tones of a base color.

```
ColorSwatch myColorSwatch =  
    MaterialColor(0xFF007AFF, {  
        50: Color(0xFF007AFF),  
        100: Color(0xFF007AFF),  
        200: Color(0xFF007AFF),  
        300: Color(0xFF007AFF),  
        400: Color(0xFF007AFF),  
        500: Color(0xFF007AFF),  
        'arrow': Color(0xFF20877E),  
        'border': Color(0xFF20877E),  
    });
```

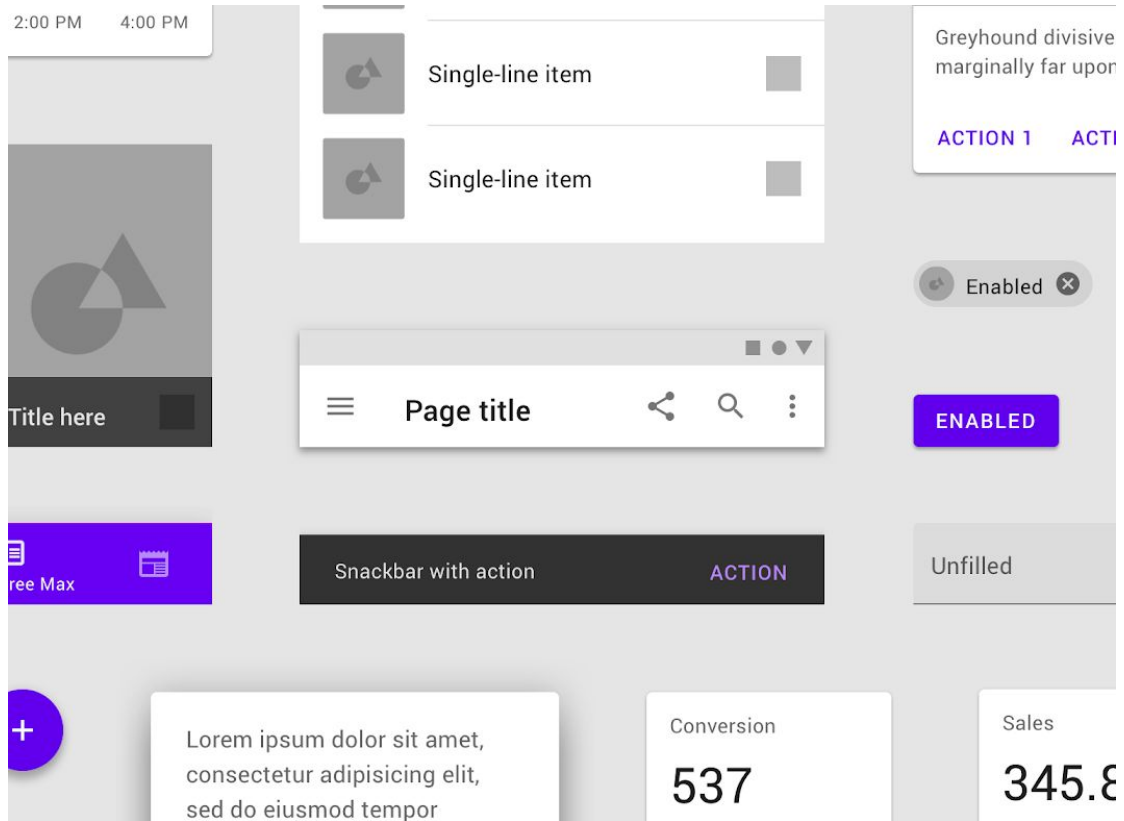
```
Container(  
    // Access a specific shade  
    color: myColorSwatch[500], )  
  
border: Border.all(  
    color: myColorSwatch['border'],  
),
```

Material Design and Themes

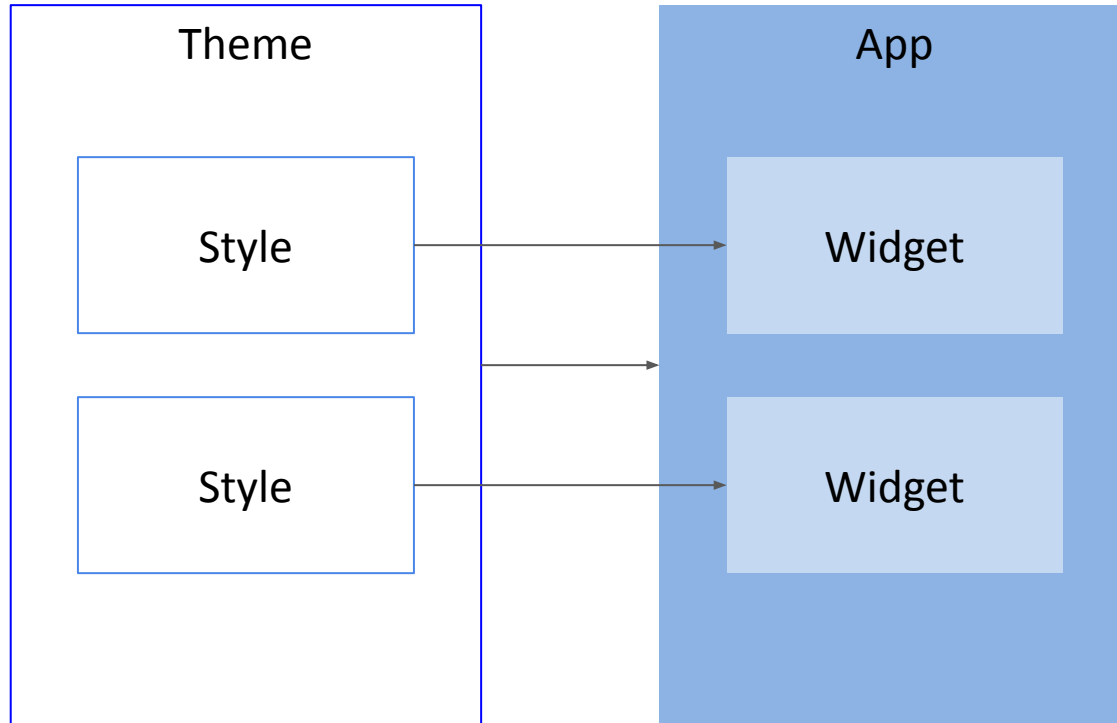
Material Design

Material Design is an open-source design system built and supported by Google designers and developers.

Default UI design for Flutter app.



Styles and Themes



Themes

To share colors and font styles throughout an app.

You can define app-wide themes.

You can extend a theme to change a theme style for one component.

Each theme defines the colors, type style, and other parameters applicable for the type of Material component.

Themes

Flutter applies styling in the following order:

1. Styles applied to the specific widget.
2. Themes that override the immediate parent theme.
3. Main theme for the entire app.

Themes

Most instances of ThemeData set values for the following two properties. These properties affect the entire app.

- colorScheme defines the colors.
- textTheme defines text styling.

```
MaterialApp(  
  title: appName,  
  theme: ThemeData(  
    useMaterial3: true,  
  
    // Define the default brightness and colors.  
    colorScheme: ColorScheme.fromSeed(  
      seedColor: Colors.purple,  
      // ...  
      brightness: Brightness.dark,  
    ),  
  
    // Define the default `TextTheme`. Use this to specify the default  
    // text styling for headlines, titles, bodies of text, and more.  
    textTheme: TextTheme(  
      displayLarge: const TextStyle(  
        fontSize: 72,  
        fontWeight: FontWeight.bold,  
      ),  
      // ...  
      titleLarge: GoogleFonts.oswald(  
        fontSize: 30,  
        fontStyle: FontStyle.italic,  
      ),  
      bodyMedium: GoogleFonts.merriweather(),  
      displaySmall: GoogleFonts.pacifico(),  
    ),  
  ),  
),
```

Using GoogleFonts
from pub.dev package

```
Container(  
  padding: const EdgeInsets.symmetric(  
    horizontal: 12,  
    vertical: 12,  
  ),  
  color: Theme.of(context).colorScheme.primary,  
  child: Text(  
    'Text with a background color',  
    // ...  
    style: Theme.of(context).textTheme.bodyMedium!.copyWith(  
      color: Theme.of(context).colorScheme.onPrimary,  
    ),  
  ),  
)
```

Override a Theme

To override the overall theme in part of an app, wrap that section of the app in a **Theme** widget.

You can override a theme in two ways:

1. Create a unique **ThemeData** instance.
2. Extend the parent theme.

Set a unique ThemeData instance

If you want a component of your app to ignore the overall theme, create a ThemeData instance. Pass that instance to the Theme widget.

```
Theme(  
  // Create a unique theme with `ThemeData`.  
  data: ThemeData(  
    colorScheme: ColorScheme.fromSeed(  
      seedColor: Colors.pink,  
    ),  
  ),  
  child: FloatingActionButton(  
    onPressed: () {},  
    child: const Icon(Icons.add),  
  ),  
);
```

Fonts and Typography

Typography covers the style and appearance of fonts.

Font Style:

1. **Typeface**: a set of common character rules. E.g. Roboto
2. **Size**: E.g. Size 18
3. **Style**: E.g. Regular, Italic
4. **Weight**: E.g. Normal, Bold

Fonts and Typography

TextTheme could be apply to:

- Display
- Headline
- Title
- Label
- Body

Size Variation:

- Small
- Medium
- Large

Display Large

Display Medium

Display Small

Headline Large

Headline Medium

Headline Small

Title Large

Title Medium

Title Small

Label Large

Label Medium

Label Small

Body Large

Body Medium

Body Small

Google Fonts Tester

<https://fonts.google.com/>

A

Fonts

🌐

Noto

🔍

Icons

📖

Knowledge

?

FAQ

 Google Fonts



A

Specimen

A

Type tester

A

Glyphs

i

About

📄

License

Get font

Roboto

Designed by [Christian Robertson](#)

Select preview text:

Writing system

Language

Heading

-

48px

+

Regular 400

I

A















Whereas recognition of the inherent dignity

No one shall be subjected to arbitrary arrest, detention or exile. Everyone is entitled in full equality to a fair and public hearing by an independent and impartial tribunal, in the

Serif vs Sans-Serif

Serif

Abc



Sans-Serif

Abc

Serif vs Sans-Serif

Which font type is easier to read?

Serif 30

Serif 24

Serif 18

Serif 14

Serif 12

Sans-Serif 30

Sans-Serif 24

Sans-Serif 18

Sans-Serif 14

Sans-Serif 12

Design for Everyone

Design for Everyone

Design for Everyone is a design philosophy that aims to create products and services that are **accessible** and **usable** by as many people as possible, regardless of their **age**, **ability**, or **background**.

In the context of **mobile app development**, it means making your app accessible to as many people as possible, regardless of their **ability**, **language**, or **device**.

Design for Everyone

Methods:

1. Internationalizing
2. Supporting different screens
3. Make apps more accessible

Internationalizing

Internationalizing

Internationalizing your app involves making it adaptable to different **languages**, **cultures**, and **regions**.

1. Language Translation
2. Generate Localization Delegates
3. Date, Number and Currency Formatting
4. Text Direction
5. Cultural Nuances

Setting up

Then import the `flutter_localizations` library and specify `localizationsDelegates` and `supportedLocales` for your `MaterialApp` or `CupertinoApp`:

```
import 'package:flutter_localizations/flutter_localizations.dart';  
import 'package:intl/intl.dart' as Intl;
```

Setting up

The `pubspec.yaml` file has the following entries:

```
dependencies:  
  flutter:  
    sdk: flutter  
  flutter_localizations:  
    sdk: flutter  
intl: any
```

Setting up

```
return const MaterialApp(  
  title: 'Localizations Sample App',  
  localizationsDelegates: [  
    GlobalMaterialLocalizations.delegate,  
    GlobalWidgetsLocalizations.delegate,  
    GlobalCupertinoLocalizations.delegate,  
  ],
```

```
  supportedLocales: [  
    Locale('en'), // English  
    Locale('es'), // Spanish  
  ],  
  home: MyHomePage(),  
);
```

Language Translation

Create .arb files (Android Resource Bundle) for each language.

Use the `generate_localized_json` package to generate Dart code from the .arb files.

```
// en.arb - English
{
  "helloWorld": "Hello, world!",
  "greeting": "Hello, {name}!"
}
```

```
//fr.arb - French
{
  "helloWorld": "Bonjour, le monde!",
  "greeting": "Bonjour, {name}!"
}
```

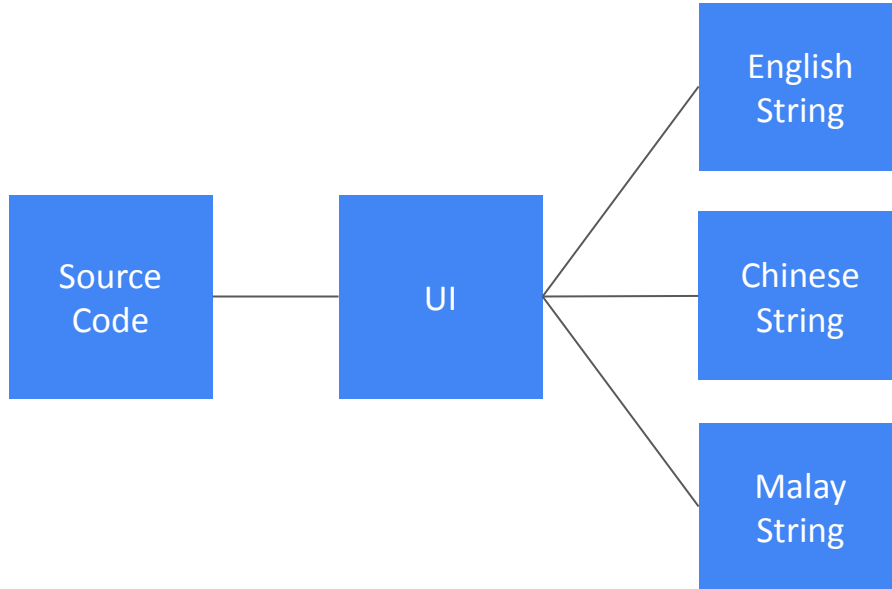
Language Translation

Mark translatable strings: Use `Intl.message()` to mark strings that need translation.

```
Text(Intl.message('Hello, world!')),
```

Create Locale Directories and String Files

At runtime, the system uses the appropriate set of string resources based on the locale currently set for the user's device.



Local Currency Symbol

```
final myCurrency = Intl.NumberFormat('#,##0.00', 'ms_MY');
```

```
symbol = myCurrency.currencySymbol;
```

```
output = myCurrency.format(total);
```

ISO 639 Language code

Format: [language code]_[country code]

Language	Language Code	Country Code	E.g.
US English	en	US	en_US
Malay (Malaysia)	ms	MY	ms_MY
Chinese (China)	zh	CN	zh_CN

ISO Language Code, https://en.wikipedia.org/wiki/List_of_ISO_639_language_codes

ISO Country Code, https://en.wikipedia.org/wiki/List_of_ISO_3166_country_codes

Supporting Different Screens

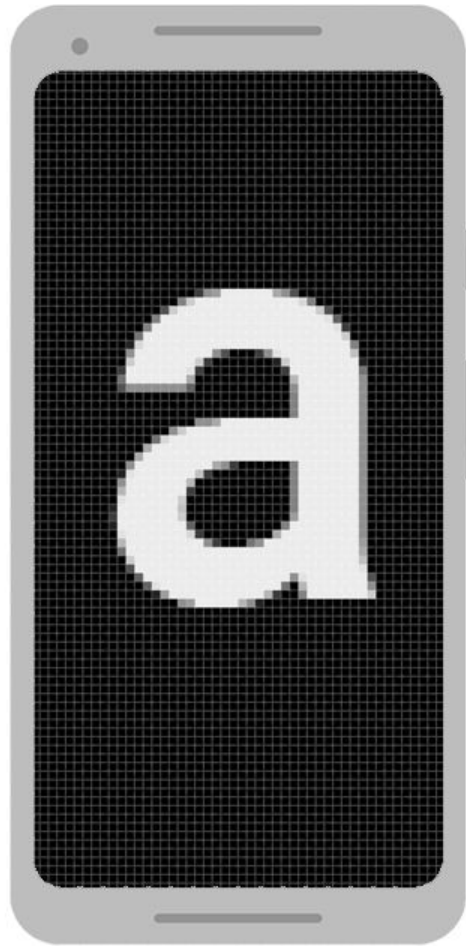
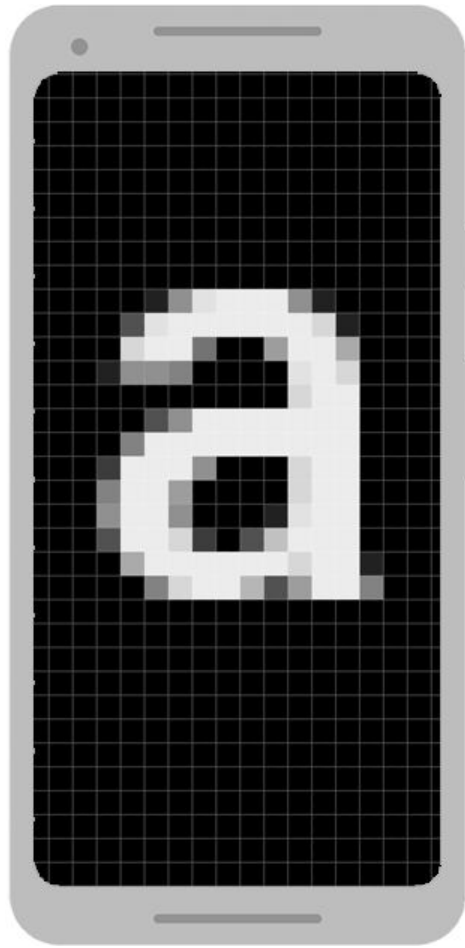
Supporting Different Screens

Categories of mobile device screens:

Size
Small
Normal
Large
xLarge

Density
Low (ldpi)
Medium (mdpi)
High (hdpi)
Extra high (xhdpi)





Supporting Different Screens

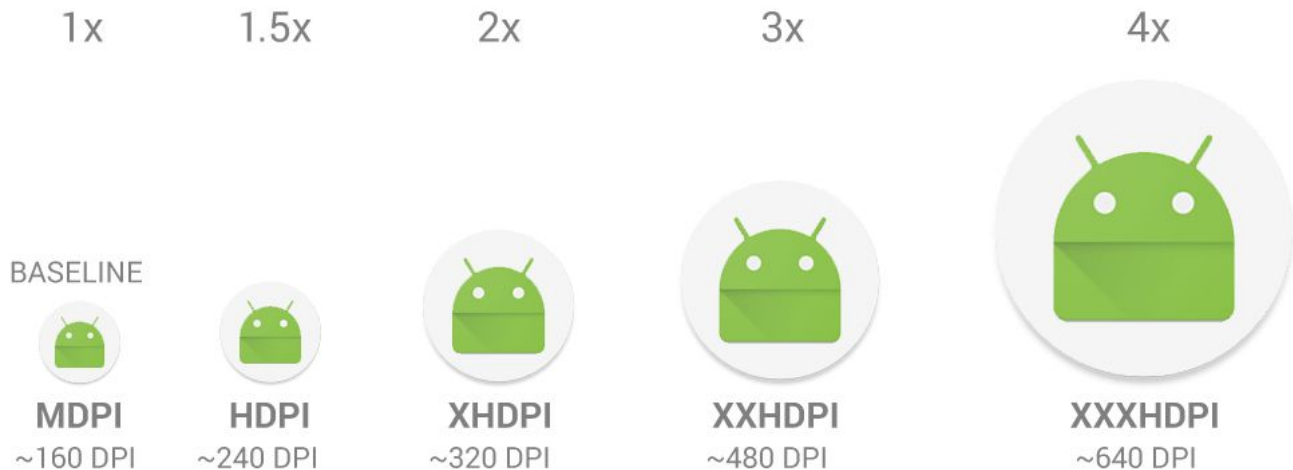
Adaptive Design

1. Utilize `Flexible` and `Expanded`: These widgets help you distribute space among child widgets.
2. Responsive Images: Use `FittedBox` to scale images to fit the available space.

Supporting Different Screens

Platform-Specific Considerations

In the context of Android and iOS, you can provide alternative bitmap resources for good graphical quality and performance



Supporting Different Screens

Create a directory named `assets` in your Flutter project's root directory.

Within this directory, create subfolders for each resolution: 1x, 2x, 3x, etc.

```
assets/  
  images/  
    logo.png  
    logo@2x.png  
    logo@3x.png
```

Supporting Different Screens

Add the `assets` section to your `pubspec.yaml` file:

```
flutter:  
  assets:  
    - assets/images/logo.png
```

Supporting Different Screens

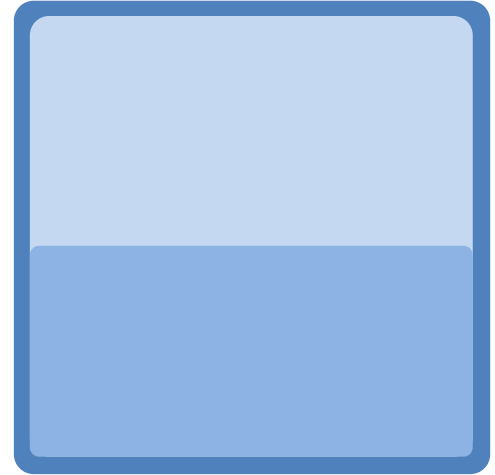
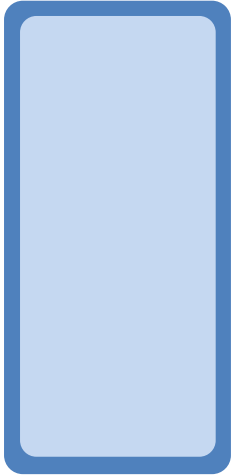
Use the `Image.asset()` widget to load the appropriate asset based on the device's screen density:

```
Image.asset('assets/images/logo.png'),
```

The system selects the most suitable asset based on the device's pixel density.

Supporting Different Screens

You should also consider screen orientation and slit screen.



Supporting Different Screens

You can build different layouts depending on a given Orientation.

```
import 'package:flutter/material.dart';

class MyWidget extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: MediaQuery.of(context).orientation ==
        Orientation.portrait ? PortraitLayout() : LandscapeLayout(),
    );
  }
}
```

Supporting Different Screens

To force your app to a specific orientation, you can use the `SystemChrome` class:

```
void main() {  
  WidgetsFlutterBinding.ensureInitialized();  
  SystemChrome.setPreferredOrientations([  
    DeviceOrientation.portraitUp,  
  ]);  
  runApp(const MyApp());  
}
```

Support Display Cutouts



Tear
Drop



Notch



Punch Hole
(Middle)



Punch Hole
(Left)

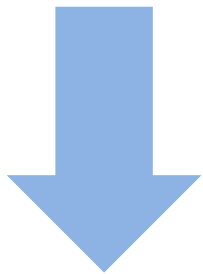


Punch Hole
(Right)

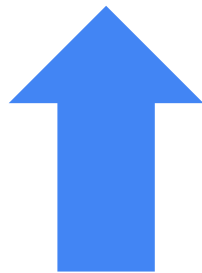
Support Display Cutouts



Specify Minimum and Target API Levels



minSdkVersion = the lowest API level with which your app is compatible



targetSdkVersion = the highest API level against which you've designed and tested your app

Specify Minimum and Target API Levels

In the Project settings:

```
android {  
    . . .  
  
    defaultConfig {  
        . . .  
        minSdkVersion 16  
        targetSdkVersion 34  
        . . .  
    }  
}
```

To support more Android
devices

Specify Minimum and Target API Levels

In the Project settings:

```
android {  
    . . .  
  
    defaultConfig {  
        . . .  
        minSdkVersion 16  
        targetSdkVersion 34  
        . . .  
    }  
}
```



To include the latest OS
features

Make Apps More Accessible

Accessibility

Regardless of ability, users are able to navigate, understand, and use an app successfully

Consideration:



Navigation



Readability



Guidance and Feedback

Accessibility - Navigation

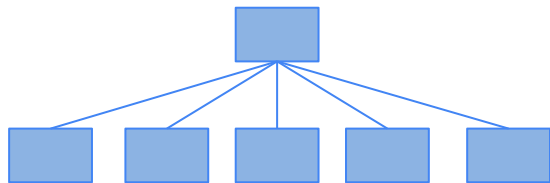
1. Support screen readers by Semantics.

```
Semantics(  
  label: 'This is a button',  
  child: ElevatedButton(  
    onPressed: () {},  
    child: Text('Click Me'),  
  ),  
),
```

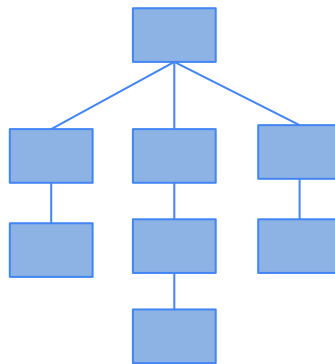
Accessibility - Navigation

2. Create easy-to-follow navigation

- Support keyboards or gestures input
- Avoid having UI elements fade out or disappear after a certain amount of time
- Create flat navigation structure



Do



Don't

Accessibility - Navigation

3. Make touch targets large

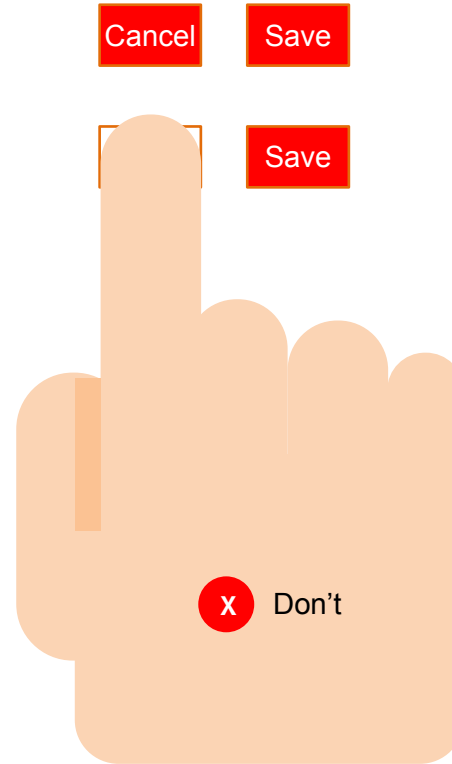
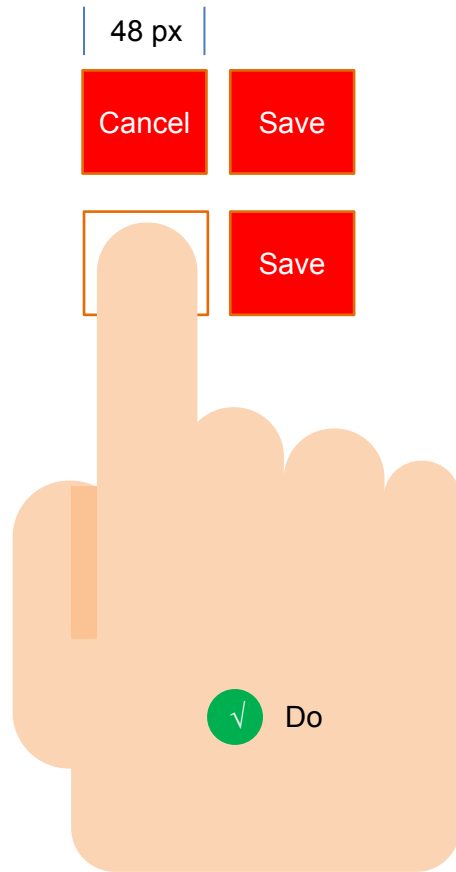
- Min size 48 x 48 pixel
- Space between elements min 8dp



Do

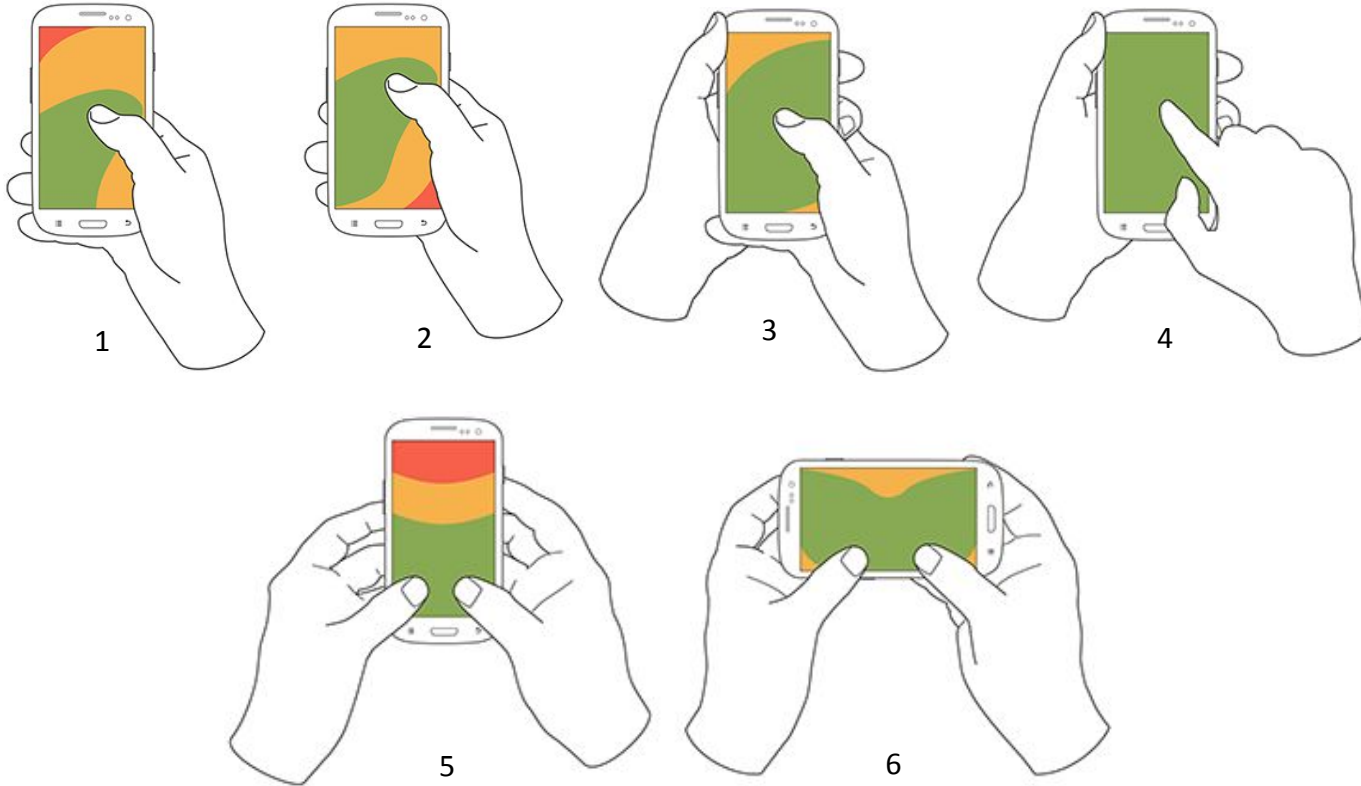


Don't

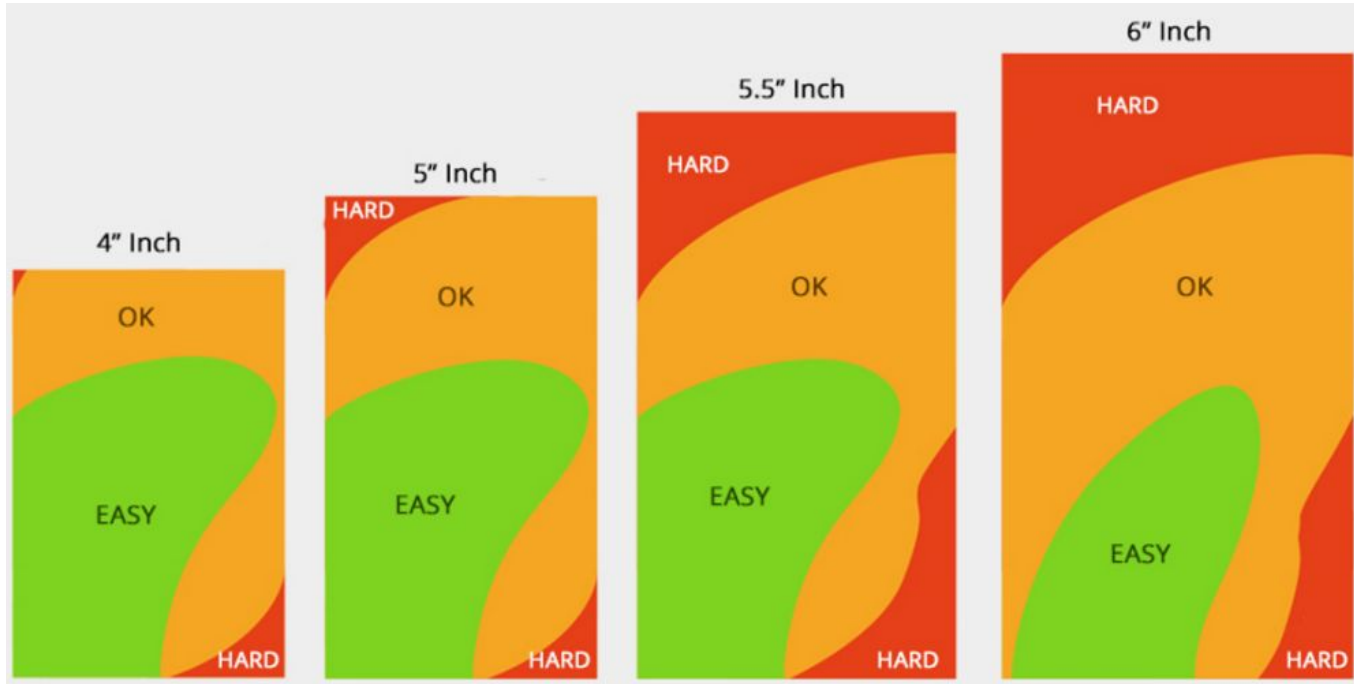


Index finger fits snugly inside.
Target edges give visual feedback.

How users hold their phones?



Touch Zone



<https://www.probeseven.com/blog/mobile-uiux-design-considerations-outdoors-visually-challenged/>

Gesture Navigation

Android 10 (API 29) supports fully gestured-based navigation

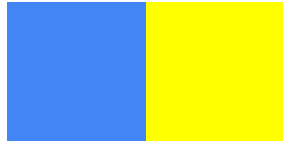
Ensure your apps:

- Extend content from edge to edge
- Handle conflicting gestures



Accessibility - Readability

4. Provide adequate colour contrast

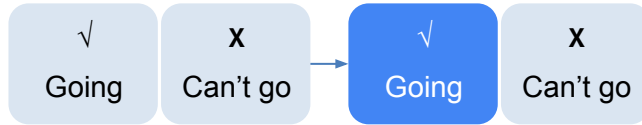


✓ Do



✗ Don't

Use more than just colour to convey information



✓ Do



✗ Don't

The Material Design Tools

<https://m2.material.io/design/color/the-color-system.html#tools-for-picking-colors>

Red 50	#FFEBEE
100	#FFCDD2
200	#EF9A9A
300	#E57373
400	#EF5350
500	#F44336
600	#E53935
700	#D32F2F
800	#C62828
900	#B71C1C
A100	#FF8A80

Pink 50	#FCE4EC
100	#F8BBD0
200	#F48FB1
300	#F06292
400	#EC407A
500	#E91E63
600	#D81B60
700	#C2185B
800	#AD1457
900	#880E4F
A100	#FF80AB

Purple 50	#F3E5F5
100	#E1BEE7
200	#CE93D8
300	#BA68C8
400	#AB47BC
500	#9C27B0
600	#8E24AA
700	#7B1FA2
800	#6A1B9A
900	#4A148C
A100	#EA80FC

Accessibility - Readability

5. Make media content more accessible

- Include controls for users to pause or stop video and audio files
- Provide transcript/caption

Accessibility - Guidance and Feedback

6. Make interactive controls clear and discoverable

- Interactive controls have text labels, tooltips, or placeholder text to indicate their purpose
- When naming elements, be consistent in your terminology throughout your app.

Find Out More

Colors, <https://api.flutter.dev/flutter/dart-ui/Color-class.html>

Themes, <https://docs.flutter.dev/cookbook/design/themes>

Internationalization,
<https://docs.flutter.dev/ui/accessibility-and-internationalization/internationalization>